# Chapter 23: XML

An Extensible Markup Language file, XML, is any flat file which conforms to a certain set of well-defined rules. In the first half of this chapter we are going to learn about those rules and the vocabulary used with XML files. In the second half of this chapter, we will learn how to read from and write to XML files from within C#.

Before we begin however a reasonable question to ask is why should we use XML files at all when we have perfectly good data storage with our relational databases?

Four reasons:

- First they are much easier to review and understand by humans, unless you are a database person relational databases can be pretty opaque.

- Second, they are super flexible and very easy to manipulate from within C#.

- Third, they are a great alternative when you don't have immediate access to a database. For example, take a delivery truck driver, they work all day recording their deliveries to an XML file only to offload that file to the relational database when they return to the loading dock.

- Finally, because they are free!

So, if XML files are so cool why not use them exclusively and avoid the high costs of a relational database? Because they don't support many of the valuable features found in relational databases like concurrency, journaling, availability and referential integrity.

Net, use XML files for temporary storage and bite the bullet and spend the money on a relational database for permanent storage.

Let's begin by examining the XML file shown here. (Figure 23.1) The first thing you notice is all those paired angled brackets with words in them. Those are called tags and they delineate information. The root, or base-level, tag for this document is named products. The products opening tag is designated via <products> and its corresponding closing tag is written as </products>. Our products node has five child nodes each delineated via the <product> opening tag and the </product> closing tag.

By the way, I created this file using my Notepad application. XMLNotepad is another good alternative but I find it a little cumbersome. If you open an XML file in any of the major browsers, they will color-code and format it for you. The file shown here was opened in Edge.

**Figure 23.1: XML File**

```
<products>
  <product id="1">
    <product_desc>Strawberry Jam </product_desc>
    <product_name>Grandpa's Strawberry Jam</product_name>
    <quantity_on_hand>10</quantity_on_hand>
    <product_length>15</product_length>
    <product_width>20</product_width>
  </product>
  <product id="2">
    <product_desc>Strawberry Jelly </product_desc>
    <product_name>Grandpa's Strawberry Jelly</product_name>
    <quantity_on_hand>100</quantity_on_hand>
    <product_length>15</product_length>
    <product_width>20</product_width>
  </product>
  <product id="3">
    <product_desc>Blueberry Jam </product_desc>
    <product_name>Grandpa's Blueberry Jam</product_name>
    <quantity_on_hand>10</quantity_on_hand>
    <product_length>15</product_length>
    <product_width>20</product_width>
  </product>
  <product id="4">
    <product_desc>Blueberry Jelly </product_desc>
    <product_name>Grandpa's Blueberry Jelly</product_name>
    <quantity_on_hand>10</quantity_on_hand>
    <product_length>15</product_length>
    <product_width>20</product_width>
  </product>
  <product id="5">
    <product_desc>Shine</product_desc>
    <product_name>Grandma's Shine</product_name>
    <quantity_on_hand>10</quantity_on_hand>
    <product_length>15</product_length>
    <product_width>20</product_width>
  </product>
</products>
```

Now for a little more XML vocabulary. An XML tag is that pair of angled brackets which delineate some information. An XML node is any XML unit of content and an XML element is an opening tag, a closing tag and all the content between those tags.

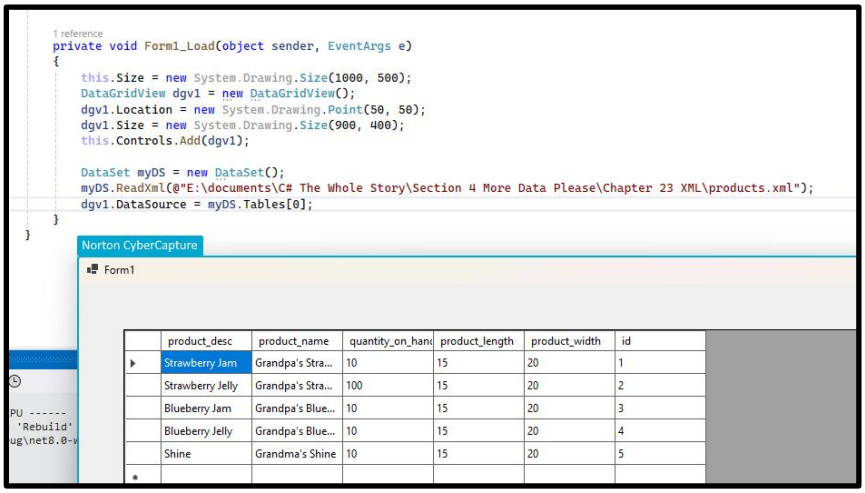
So, what is the difference then between a node and an element? All elements are nodes, but not all nodes are elements. For example, comments are a node, but they are not an element.

Be calm, this vocabulary lesson is nearly complete. Each of our product opening tags has a corresponding id attribute. XML attributes are identified by an opening label followed by an equal sign and a string literal delineated via double quotes; e.g. id = “1”. Each opening tag may have zero, one or many attributes. Closing tags may not contain attributes.

Oh, by the way, XML tags are case sensitive just like class names in C#. As such <Product> does not serve as a valid opening tag to <product>. When tags are improperly paired the XML file is said to be not well-formed. C#, Oracle and SQL Server can only work with well-formed XML files so be careful.

We are now ready to write some C# code against our products.xml file. In my form load routine, I instantiate a DataGridView and size it and the form. I then write a sum total of three lines of code to populate the DataSource of that grid view. How easy is that? (Figure 23.2)

**Figure 23.2: ReadXML**

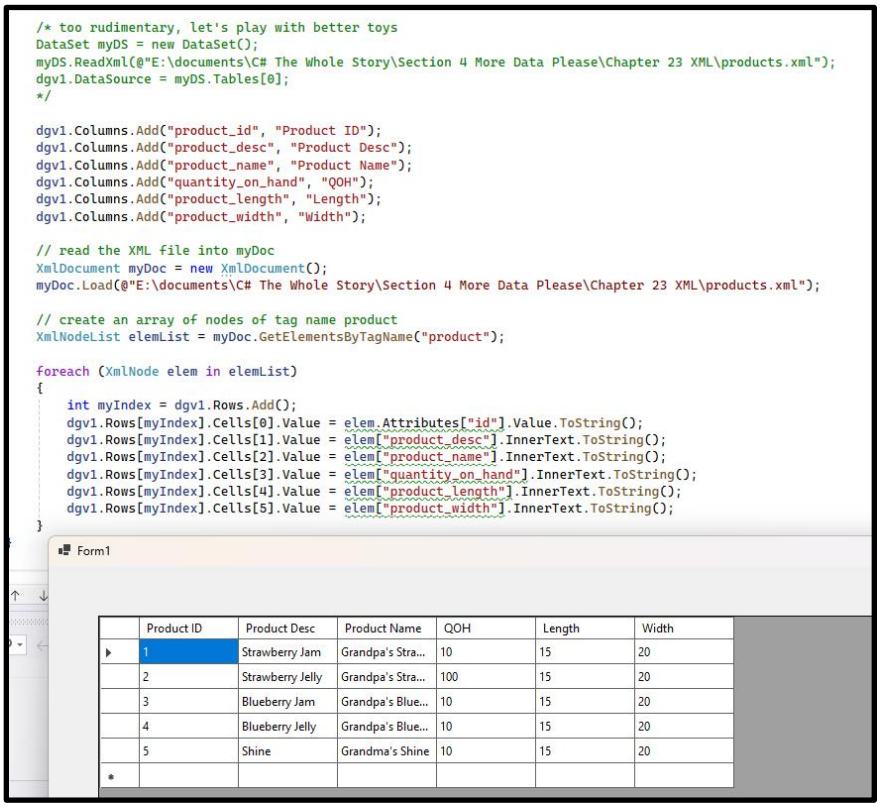



Well as is often the case, if it sounds too good to be true it often isn't. Well, that is sort of the case here because I can't use this strategy to make updates back to the external XML file. Net, this super easy technique is pretty useless. We of course could use a foreachrow loop on the DataSet but that would be a lot of unnecessary updates for most applications.

So how do we improve on the ReadXML syntax? The answer comes in the form of the XMLDocument class shown here. (Figure 23.3) Again pretty straightforward method for reading our XML file via the XML Load event. This time however, I have to explicitly declare my data grid columns because we don't have that information available from within a DataSet object.

I next create an instance of the XMLNodeList class named elemList via the GetElementsByTagName method of our XMLDocument object.
The foreach loop is then straightforward. Use each occurrence of the XMLNode class in our XMLNodeList object to populate our data grid. Please note the reference to the Attributes collection as well as the use of the InnerText property of each XMLNode object.

**Figure 23.3: XMLDocument class**



```
/* too rudimentary, let's play with better toys
DataSet myDS = new DataSet();
myDS.ReadXml(@"E:\documents\C# The Whole Story\Section 4 More Data Please\Chapter 23 XML\products.xml");
dgv1.DataSource = myDS.Tables[0];
*/

dgv1.Columns.Add("product_id", "Product ID");
dgv1.Columns.Add("product_desc", "Product Desc");
dgv1.Columns.Add("product_name", "Product Name");
dgv1.Columns.Add("quantity_on_hand", "QOH");
dgv1.Columns.Add("product_length", "Length");
dgv1.Columns.Add("product_width", "Width");

// read the XML file into myDoc
XmlDocument myDoc = new XmlDocument();
myDoc.Load(@"E:\documents\C# The Whole Story\Section 4 More Data Please\Chapter 23 XML\products.xml");

// create an array of nodes of tag name product
XmlNodeList elemList = myDoc.GetElementsByTagName("product");

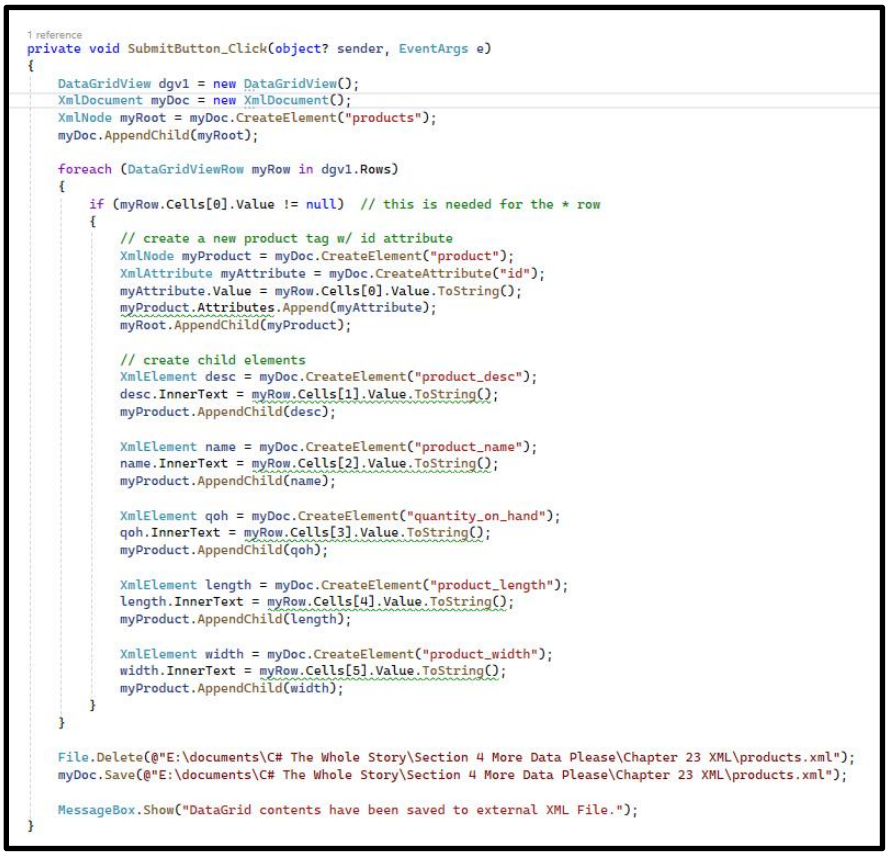
foreach (XmlNode elem in elemList)
{
    int myIndex = dgv1.Rows.Add();
    dgv1.Rows[myIndex].Cells[0].Value = elem.Attributes["id"].Value.ToString();
    dgv1.Rows[myIndex].Cells[1].Value = elem["product_desc"].InnerText.ToString();
    dgv1.Rows[myIndex].Cells[2].Value = elem["product_name"].InnerText.ToString();
    dgv1.Rows[myIndex].Cells[3].Value = elem["quantity_on_hand"].InnerText.ToString();
    dgv1.Rows[myIndex].Cells[4].Value = elem["product_length"].InnerText.ToString();
    dgv1.Rows[myIndex].Cells[5].Value = elem["product_width"].InnerText.ToString();
}
```

Form1

| | Product ID | Product Desc | Product Name | QOH | Length | Width |
|---|---|---|---|---|---|---|
| ▶ | 1 | Strawberry Jam | Grandpa's Stra... | 10 | 15 | 20 |
| | 2 | Strawberry Jelly | Grandpa's Stra... | 100 | 15 | 20 |
| | 3 | Blueberry Jam | Grandpa's Blue... | 10 | 15 | 20 |
| | 4 | Blueberry Jelly | Grandpa's Blue... | 10 | 15 | 20 |
| | 5 | Shine | Grandma's Shine | 10 | 15 | 20 |
| * | | | | | | |

I next went back to my GUI and added a submit Button. The click event for that guy writes the contents of our DataGrid back out to an XML file. Pretty easy code once you see an example. (Figure 23.4)

**Figure 23.4: Write Grid Contents to an XML File**



```
1 reference
private void SubmitButton_Click(object? sender, EventArgs e)
{
    DataGridView dgv1 = new DataGridView();
    XmlDocument myDoc = new XmlDocument();
    XmlNode myRoot = myDoc.CreateElement("products");
    myDoc.AppendChild(myRoot);

    foreach (DataGridViewRow myRow in dgv1.Rows)
    {
        if (myRow.Cells[0].Value != null)  // this is needed for the * row
        {
            // create a new product tag w/ id attribute
            XmlNode myProduct = myDoc.CreateElement("product");
            XmlAttribute myAttribute = myDoc.CreateAttribute("id");
            myAttribute.Value = myRow.Cells[0].Value.ToString();
            myProduct.Attributes.Append(myAttribute);
            myRoot.AppendChild(myProduct);

            // create child elements
            XmlElement desc = myDoc.CreateElement("product_desc");
            desc.InnerText = myRow.Cells[1].Value.ToString();
            myProduct.AppendChild(desc);

            XmlElement name = myDoc.CreateElement("product_name");
            name.InnerText = myRow.Cells[2].Value.ToString();
            myProduct.AppendChild(name);

            XmlElement qoh = myDoc.CreateElement("quantity_on_hand");
            qoh.InnerText = myRow.Cells[3].Value.ToString();
            myProduct.AppendChild(qoh);

            XmlElement length = myDoc.CreateElement("product_length");
            length.InnerText = myRow.Cells[4].Value.ToString();
            myProduct.AppendChild(length);

            XmlElement width = myDoc.CreateElement("product_width");
            width.InnerText = myRow.Cells[5].Value.ToString();
            myProduct.AppendChild(width);
        }
    }

    File.Delete(@"E:\documents\C# The Whole Story\Section 4 More Data Please\Chapter 23 XML\products.xml");
    myDoc.Save(@"E:\documents\C# The Whole Story\Section 4 More Data Please\Chapter 23 XML\products.xml");

    MessageBox.Show("DataGrid contents have been saved to external XML File.");
}
```

Okay, that concludes this brief overview of reading and writing to an XML file from within C#. I told you it was easy. It also completes this section on data storage and management. In our next, and final section, we will learn how to deploy a C# application.